TECHNISCHE UNIVERSITÄT
CHEMNITZ

Professur Betriebssysteme

Algorithmen und Programmierung, WS 2025/2026

Belegaufgabe #3: **Perlenkette**

Deadline: **01. Dezember 2025, 23.59 Uhr**
Erreichbare Punkte: **6**

1. Motivation

Ziel dieser Aufgabe ist es, …

- … Invariante(n) in mathematischen Problemstellungen zu identifizieren;
- … einen rekursiven Algorithmus zu entwerfen;
- … Daten in Arrays zu verwalten;
- … einen Trait zu implementieren.

2. Aufgabenstellung

2.1 Perlenketten und Ihre Unterscheidung

Sie wollen einen Handel für Perlenketten eröffnen. Die von Ihnen gehandelten Ketten haben zwar alle eine gleichlange Schnur, auf die maximal 20 Perlen passen, aber die tatsächliche Anzahl von Perlen und ihre Kombination kann recht unterschiedlich sein. Die Perlen sind zwar alle gleich groß, unterscheiden sich jedoch in ihren Farben. Wir wollen die Farben hier als RGB-Wert[1] durch Dreiertuple von `u8` darstellen. Abbildung 1 zeigt ein Beispiel für eine Perlenkette aus vier Perlen.

Sie wollen ausschließlich einzigartige Perlenketten in ihrem Besitz haben. Daher ist es für den An- und Verkauf relevant, ob eine Kette einer bestimmten Art bereits in Ihrem Besitz ist oder nicht. Sie schreiben ein Rust-Programm für die automatisierte Behandlung dieser Fragestellung. Sie haben bereits einige Überlegungen für die Darstellung von Perlenketten mit *bis zu* 20 Perlen angestellt:

- Jede Perlenkette wird innerhalb eines `[(u8,u8,u8); 20]`-Arrays dargestellt. Jedes Array-Element beschreibt entweder die Farbe einer Perle, oder gibt mit `(0,0,0)` an, dass das Array-Element nicht für die Darstellung der Kette genutzt wird. [2]

[1] `https://de.wikipedia.org/wiki/RGB-Farbraum`
[2] Sie handeln nicht mit schwarzen Perlen, weil diese unter Naturschutz stehen.

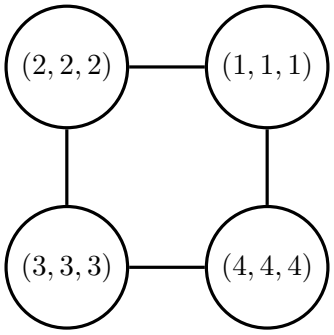

Abbildung 1: Beispiel einer Perlenkette.

- Das Array ist als ein Ring aufzufassen. Somit wird eine Sequenz von Perlen, welche über das Ende des Arrays hinausgeht, am Anfang des Arrays fortgesetzt (siehe Abbildung 2e). Mit anderen Worten: Perlen hängen zusammen wenn
  - sie im Array benachbart sind, d.h. sich ihr Indexwert um 1 unterscheidet;
  - zwischen ihnen nur Lücken (also `(0,0,0)`) oder Arrayanfang und -ende sind.

Beachten Sie, dass unterschiedliche Belegungen des Arrays die gleiche Perlenkette darstellen *können*. Eine Array-Darstellung kann, im Vergleich zu einer weiteren Darstellung, gespiegelt, rotiert und/oder verschoben sein und trotzdem *können* beide die gleiche Perlenkette darstellen. Zum Beispiel stellen alle Array-Belegungen aus Abbildung 2 die gleiche Perlenkette aus Abbildung 1 dar.

Es gibt jedoch eine Einschränkung in der Darstellung: es darf *nur eine* zusammenhängende Lückensequenz geben, d h. alle leeren Plätze in dem Array müssen (möglicherweise über die Array-Grenzen hinweg) aufeinanderfolgen. Ist dies nicht gegeben, ist die Kettenbeschreibung ungültig (nicht *valide*). Eine leere Kette, also ein Array, das ausschließlich aus $(0, 0, 0)$-Elementen besteht, gilt ebenfalls als valide.

2.2 Aufgabe

Implementieren Sie den folgenden Trait für den Datentyp `[(u8,u8,u8);20]`.

```
trait Necklace {
    fn get_len(self) -> usize;
    fn is_valid(self) -> bool;
    fn is_identical_with(self, other: Self) -> bool;
}
```

Die Methoden sollen dabei die folgende Funktionalität haben:

- `is_valid(self)`: Gibt `true` zurück, falls die Repräsentation `self` einer Halskette valide ist.
- `get_len(self)`: Gibt die Länge der in `self` dargestellten Halskette zurück. Falls die Repräsentationen der Halskette nicht gültig ist, soll 0 zurückgegeben werden.

	$i = 10$	$i = 11$	$i = 12$	$i = 13$	$i = 14$	$i = 15$	
$\cdots$	$(0,0,0)$	$(1,1,1)$	$(2,2,2)$	$(3,3,3)$	$(4,4,4)$	$(0,0,0)$	$\cdots$

(a) Eins-zu-eins Darstellung ab Index 10 im Feld.

	$i = 12$	$i = 13$	$i = 14$	$i = 15$	$i = 16$	$i = 17$	
$\cdots$	$(0,0,0)$	$(1,1,1)$	$(2,2,2)$	$(3,3,3)$	$(4,4,4)$	$(0,0,0)$	$\cdots$

(b) Eins-zu-eins Darstellung ab Index 12 (die Darstellung aus Abbildung 2a wurde *verschoben*).

	$i = 10$	$i = 11$	$i = 12$	$i = 13$	$i = 14$	$i = 15$	
$\cdots$	$(0,0,0)$	$(3,3,3)$	$(4,4,4)$	$(1,1,1)$	$(2,2,2)$	$(0,0,0)$	$\cdots$

(c) Um zwei nach rechts *rotierte* Darstellung.

	$i = 10$	$i = 11$	$i = 12$	$i = 13$	$i = 14$	$i = 15$	
$\cdots$	$(0,0,0)$	$(3,3,3)$	$(2,2,2)$	$(1,1,1)$	$(4,4,4)$	$(0,0,0)$	$\cdots$

(d) Um zwei nach rechts rotierte und *gespiegelte* Darstellung.

$i = 0$	$i = 1$	$i = 2$		$i = n-3$	$i = n-2$	$i = n-1$
$(3,3,3)$	$(2,2,2)$	$(0,0,0)$	$\cdots$	$(0,0,0)$	$(1,1,1)$	$(4,4,4)$

(e) Darstellung einer Sequenz über die Array-Grenze hinweg (Array als Ring mit Länge $n = 20$).

Abbildung 2: *Einige mögliche Repräsenationen der Perlenkette aus Abbildung 1. Beachten Sie, dass das Array in den Unterabbildungen 2a, 2b, 2c und 2d nach links und rechts weitergeht und in diesen Beispielen mit* $(0,0,0)$ *aufgefüllt wurde. In Unterabbildung 2e wird das Array in der Mitte mit* $(0,0,0)$ *aufgefüllt.*

- `is_identical_with(self, other: Self)`: Überprüft, ob die dargestellten Halsketten in `self` und `other` identisch sind. Vorsicht: es sollen die *Halsketten* verglichen werden, die *Repräsentationen* in den Arrays können sich entsprechend der oben genannten Regeln unterscheiden! Damit eine Halskette (nicht ihre Repräsentation) einer anderen gleicht, muss sie die gleichen Farben in der gleichen Reihenfolge sowie die gleiche Anzahl Perlen umfassen.

3. Einschränkungen

Bei der Programmierung unterliegen Sie einigen Einschränkungen, d. h. Sie dürfen bestimmte Programmierkonstrukte **nicht** nutzen:[3]

[3] Wenn Sie eines der folgenden Konstrukte nicht kennen, dann ignorieren Sie es.

- Schreiben Sie **keine** `main()`-Funktion![4]
- Sie dürfen die Signaturen der Aufgabenmethoden nicht ändern.
- Die Nutzung von Funktionen aus der Standardbibliothek oder aus der Preamble sind nicht erlaubt (Sie müssen also sämtlichen Code selbst schreiben).
- Sie dürfen **keine** Schleifen nutzen.
- Sie dürfen **keine** Iteratoren nutzen.
- Sie dürfen **keinen** unsicheren Code schreiben (Schlüsselwort `unsafe`).
- Sie dürfen die Schlüsselworte `let` und `mut` nicht nutzen.
- Es dürfen **keine** Attribute genutzt werden.
- Ihr Code darf **nichts** ausgeben (`println`, etc.).

4. Hinweise zur Aufgabe

Da es sich um eine Prüfungsleistung handelt, müssen Sie die Ausgabenstellung allein und ohne fremde Hilfe bewältigen. Als unzulässige Hilfe zählen sowohl Lösungen anderer Personen als auch Lösungen von KIs oder aus dem Internet.

Die folgenden Schritte können Ihnen helfen, die Aufgabe zu bearbeiten. Falls Sie keine Hilfestellung benötigen, können Sie diesen Abschnitt überspringen.

- Entwerfen Sie einen Algorithmus (Vorgehen), der die in der Aufgabenstellung geforderte Vertauschung ausführt.
- Testen Sie diesen Algorithmus mit Papier und Bleistift mit allen obenstehenden Beispielen.
- Denken Sie sich weitere Beispiele aus und testen Sie Ihren Algorithmus auch mit diesen.
- Falls Ihr Algorithmus nicht bereits rekursiv ist, überführen Sie Ihn in einen rekursiven Algorithmus.
- Überführen Sie den rekursiven Algorithmus in die gewünschte Rust-Funktion.
- Sie dürfen eigene Hilfsfunktionen schreiben (die oben genannten Einschränkungen gelten auch hier).
- Beachten Sie in Ihrem Code den Unterschied zwischen *Rückgabe* und *Ausgabe* eines Wertes.
- Testen Sie Ihren Code (siehe nächster Abschnitt).

[4] Sie dürfen diese natürlich schreiben, aber sie soll nicht in der Abgabe enthalten sein.

5. Testen

Beachten Sie, dass Ihre Abgabe kein vollständiges Programm ist, und dass sie keine `main()`-Funktion enthalten soll. Da Sie für den Test Ihrer Funktion jedoch ein ausführbares Programm benötigen, können Sie zum Testen z.B. folgenden Code nutzen:

```rust
fn main() {
    println!(
        "{}", // sollte true sein
        [
            (1,1,1),(1,1,1),(1,1,1),(1,1,1),(1,1,1),
            (1,1,1),(1,1,1),(0,0,0),(0,0,0),(1,1,1),
            (1,1,1),(1,1,1),(1,1,1),(1,1,1),(1,1,1),
            (1,1,1),(1,1,1),(1,1,1),(1,1,1),(1,1,1),
        ].is_valid()
    );
    println!(
        "{}", // sollte 18 sein
        [
            (1,1,1),(1,2,3),(1,1,1),(1,1,1),(1,1,1),
            (1,1,1),(1,1,1),(0,0,0),(0,0,0),(1,1,1),
            (1,1,1),(1,1,1),(1,1,1),(1,1,1),(1,1,1),
            (1,1,1),(1,1,1),(1,1,1),(1,1,1),(1,1,1),
        ].get_len()
    );
    println!(
        "{}", // sollte true sein
        [
            (1,1,1),(1,2,3),(1,1,1),(1,1,1),(1,1,1),
            (1,1,1),(1,1,1),(0,0,0),(0,0,0),(1,1,1),
            (1,1,1),(1,1,1),(1,1,1),(1,1,1),(1,1,1),
            (1,1,1),(1,1,1),(1,1,1),(1,1,1),(1,1,1),
        ].is_identical_with([
                (0,0,0),(0,0,0),(1,1,1),(1,1,1),(1,1,1),
                (1,1,1),(1,1,1),(1,2,3),(1,1,1),(1,1,1),
                (1,1,1),(1,1,1),(1,1,1),(1,1,1),(1,1,1),
                (1,1,1),(1,1,1),(1,1,1),(1,1,1),(1,1,1),
            ])
    );
}
```

Beachten Sie aber, dass Ihr Abgabecode **nicht** die `main`-Funktion enthalten darf!

6. Einreichung

- Ihr Abgabe-Code muss auf den Rechnern des URZ-Pools übersetzbar sein mit dem Befehl `rustc --crate-type lib --emit obj necklace.rs`.

- Erstellen Sie ein Archiv in Form einer **.zip-Archivdatei**, welche **ausschließlich** Ihre Lösungsdatei `necklace.rs` enthält.
- Fügen Sie dem Archiv **keine** weiteren Dateien oder Ordner hinzu.
- Reichen Sie Ihre Lösung unter folgendem Link ein: `https://bildungsportal.sachsen.de/opal/auth/RepositoryEntry/41245343744/CourseNode/1757903993504206003`
- Sie erhalten eine Mail von OPAL als Bestätigung für die Einreichung Ihrer Abgabe. Bitte bewahren Sie die Mail auf.
- Falls Sie eine bestehende Lösung aktualisieren wollen, laden Sie bitte eine neue **.zip-Archivdatei** hoch. Wir bewerten Ihre letzte Abgabe (die Benennung spielt keine Rolle).

18. November 2025